Data Science: Data Cleaning

Dr. Konrad Völkel, Sommersemester 2026

Data Cleaning

Heinrich Heine
Universität Düsseldorf

Aspekte des Datenputzens:

- Ausreißer identifizieren
  - Tests
  - inhaltliche Plausibilität
- Fehlende Werte auffüllen
  - Imputation
  - Interpolation
- Datentypen identifizieren, konvertieren:
  - Datum und Uhrzeit
  - Ortsangaben
  - Zahlen
  - Kategorielle Variablen kodieren / dekodieren
  - String-Encoding

Ausreißer identifizieren

Und herausreißen

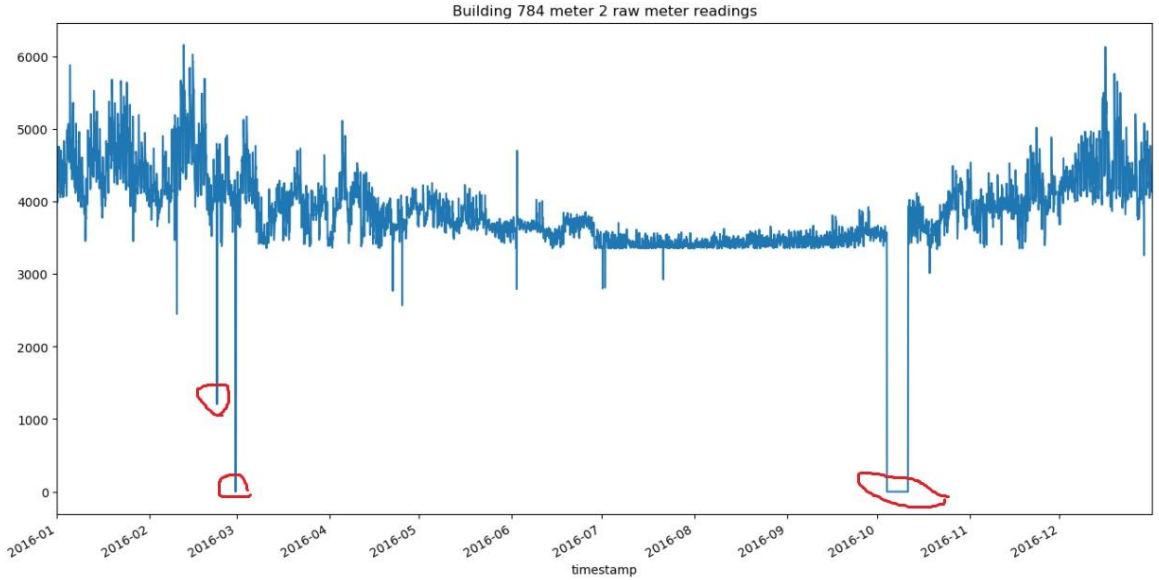


“Batman holds his smartphone to catch Pokémon”

Ausreißer identifizieren

Mit den Augen

Wenn man Daten plottet, sieht man oft schon Werte, die auffällig aus dem Spektrum herausragen - kennt man die Art der Daten, lässt sich die Plausibilität ermitteln.



Ausreißer identifizieren

Heinrich Heine
Universität Düsseldorf

Mit Pandas und Statistik

```
import numpy as np
import pandas as pd
import scipy.stats as st

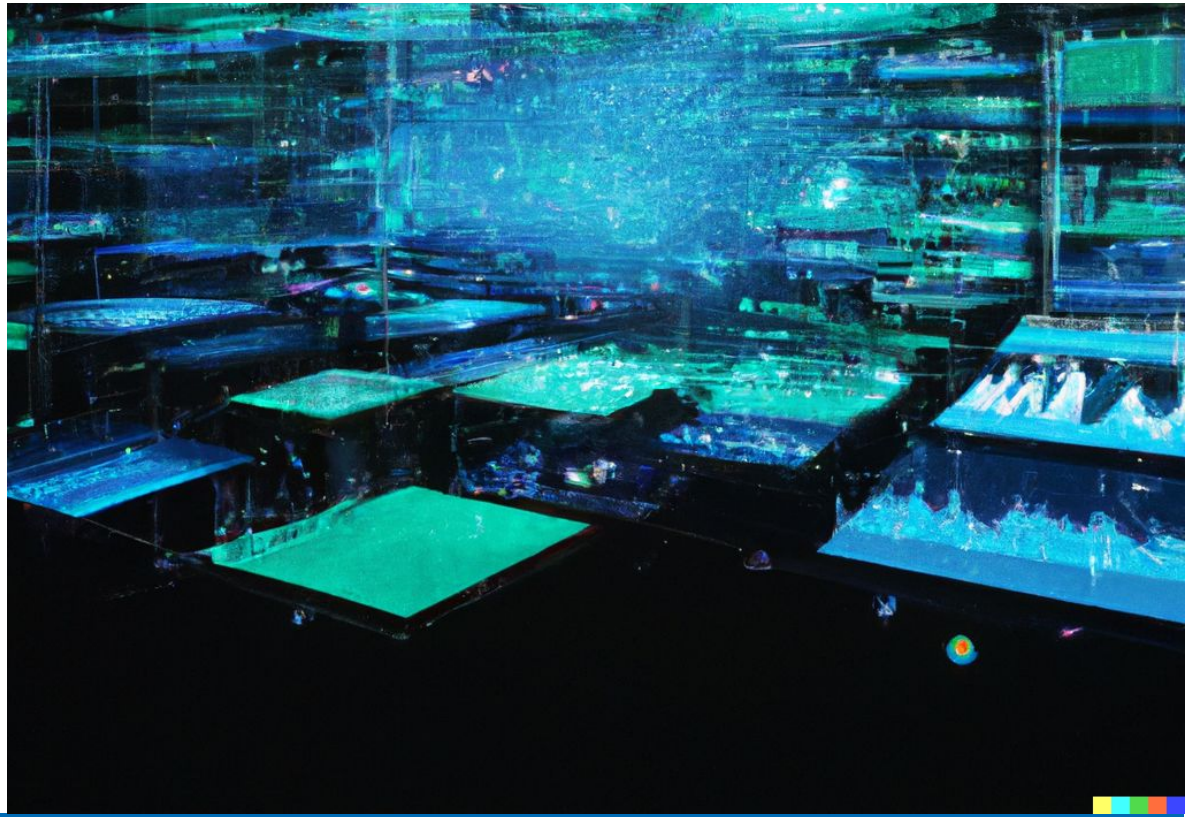
df = pd.DataFrame(...)

# Finde alles,
# was mehr als 3 Standardabweichungen vom Mittelwert entfernt ist:

outliers = df[(np.abs(st.zscore(df)) < 3).all(axis=1)]
```

Imputation

und Interpolation



“Visualizing complex data, Unreal Engine”

Imputation

Mit Pandas fehlende Daten finden

```
import numpy as np
import pandas as pd

df = pd.DataFrame(...)

df.isna()
df.notna()
```

Imputation

Mit Pandas fehlende Daten füllen

```
import numpy as np
import pandas as pd

df = pd.DataFrame(...)

# NA mit 0 ersetzen:
df.fillna(0)

# NA mit dem letzten gültigen Wert auffüllen:
df.fillna(method='pad') # pad=ffill=forward fill
df.ffill()              # äquivalent zur vorherigen Zeile
```

Imputation

Heinrich Heine
Universität Düsseldorf

Mit Pandas fehlende Daten füllen

```
import numpy as np
import pandas as pd

df = ashrae3-weather-train

air_temperature_filler = \
  pd.DataFrame(weather_df.groupby(['site_id',
                                   'day',
                                   'month'])['air_temperature'].mean(),
               columns=["air_temperature"])

weather_df.update(air_temperature_filler, overwrite=False)
```

Imputation

Mit Pandas fehlende Daten interpolieren

```
import numpy as np
import pandas as pd

df = pd.DataFrame(...)

filled_df = df.interpolate()

df.interpolate(inplace=True) # ändert df selbst
```

Strings normalisieren

in passende Datentypen konvertieren



“Cleaning up a big data mess”

Strings normalisieren

Heinrich Heine
Universität Düsseldorf

Encoding

Ein häufiges Problem ist, dass beim dekodieren eines Strings (aus einem Binärstring) ein anderes Encoding gewählt wird/wurde als beim kodieren.

Wird z.B. ‘Völkel’ mit UTF-8 kodiert und dann mit ISO-8859-1 dekodiert, kommt ‘VÃ¶lkel’ heraus. Wird umgekehrt “Völkel” mit ISO-8859-1 kodiert und mit UTF-8 dekodiert, kommt ‘V  lkel’ heraus.

Strings normalisieren

Encoding

Durchsucht man seine Strings nach ‘Ã¶’ und ‘’, findet man heraus, ob solche Probleme bestehen.

Oftmals sind dabei nur manche Strings betroffen - wenn die Daten auf unterschiedlichen Systemen erfasst oder verarbeitet wurden.

Strings normalisieren

Encoding

Probieren Sie Encoding-Fehler selbst aus:

string-functions.com/encodedecode.aspx

Strings normalisieren

Tippfehler (Typos)

Mal heisst es ‘Deutschland’, mal nur ‘Detschland’.

Systematisch Tippfehler findet man durch Gruppieren der Daten nach Levenshtein-Edit-Distanz

Geht mit dem Modul `python-Levenshtein`.

Strings normalisieren

Heinrich Heine
Universität Düsseldorf

Reguläre Ausdrücke

Reguläre Ausdrücke (regular expressions, regex) sind eine Beschreibung / Syntax für deterministische endliche Automaten, also für Sprachen vom Typ 3 (reguläre Sprachen).

Strings normalisieren

Reguläre Ausdrücke

Python hat exzellenten Regex-Support mit dem internen Modul `re`

```
myregex = re.compile(r'[a-z]+22')
myregex.match('aha22')
```

Strings normalisieren

Reguläre Ausdrücke

Nutzen Sie das VERBOSE-flag (auch ‘X’):

```
pattern = r"""
              multiline  # with comments
              Pattern
          """
actual_regex = re.compile(pattern, re.VERBOSE)
```

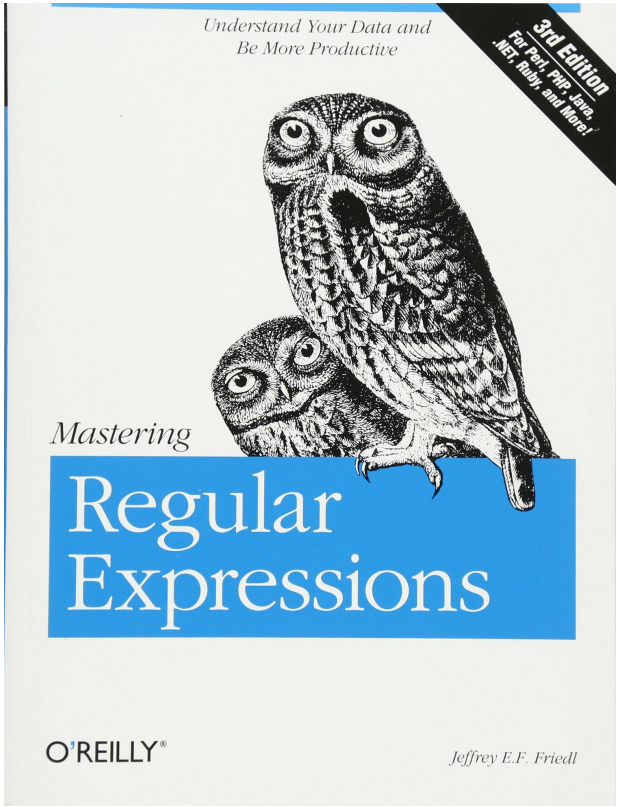
Reguläre Ausdrücke

Das BUCH

Lesen Sie das Friedl-Buch

“Mastering Regular Expressions”

(irgendwann in Ihrem Leben)



Reguläre Ausdrücke

Heinrich Heine
Universität Düsseldorf

BNF (für Perl-style Regex)

```
<RE>            ::=    <union> | <simple-RE>
<union>         ::=    <RE> "|" <simple-RE>
<simple-RE>     ::=    <concatenation> | <basic-RE>
<concatenation>::=     <simple-RE> <basic-RE>
<basic-RE>      ::=    <star> | <plus> | <elementary-RE>
<star>          ::=    <elementary-RE> "*"
<plus>          ::=    <elementary-RE> "+"
<elementary-RE>::=     <group> | <any> | <eos> | <char> | <set>
<group>         ::=    "(" <RE> ")"
<any>           ::=    "."
<eos>           ::=    "$"
<char>          ::=    any non metacharacter | "\" metacharacter
<set>           ::=    <positive-set> | <negative-set>
<positive-set>  ::=    "[" <set-items> "]"
<negative-set>  ::=    "[^" <set-items> "]"
<set-items>     ::=    <set-item> | <set-item> <set-items>
<set-item>      ::=    <range> | <char>
<range>         ::=    <char> "-" <char>
```

Reguläre Ausdrücke

Heinrich Heine
Universität Düsseldorf

BNF (für Perl-style Regex) mit Beispielen

```
<RE>            ::=    <union> | <simple-RE>
<union>         ::=    <RE> "|" <simple-RE>                    r"a|b"
<simple-RE>     ::=    <concatenation> | <basic-RE>
<concatenation>::=     <simple-RE> <basic-RE>                  r"ab"
<basic-RE>      ::=    <star> | <plus> | <elementary-RE>
<star>          ::=    <elementary-RE> "*"                     r"a*"
<plus>          ::=    <elementary-RE> "+"                     r"a+"
<elementary-RE>::=     <group> | <any> | <eos> | <char> | <set>
<group>         ::=    "(" <RE> ")"                            r"(ab|c)"
<any>           ::=    "."                                     r"ab.d"
<eos>           ::=    "$"                                     r"abc$"
<char>          ::=    any non metacharacter | "\" metacharacter
<set>           ::=    <positive-set> | <negative-set>
<positive-set> ::=     "[" <set-items> "]"                     r"[abc]"
<negative-set> ::=     "[^" <set-items> "]"                    r"[^def]"
<set-items>     ::=    <set-item> | <set-item> <set-items>
<set-item>      ::=    <range> | <char>
<range>         ::=    <char> "-" <char>                       r"[a-z]"
```

Reguläre Ausdrücke

Heinrich Heine
Universität Düsseldorf

Lazy & Greedy

Wenn wir auf

`"<em>Test</em>"`

die HTML-Tags matchen wollen - wie?

Reguläre Ausdrücke

Heinrich Heine
Universität Düsseldorf

Lazy & Greedy

Wenn wir auf

`"<em>Test</em>"`

die HTML-Tags matchen wollen:

- `r"<.+>"` matcht den ganzen String **greedy**

Reguläre Ausdrücke

Heinrich Heine
Universität Düsseldorf

Lazy & Greedy

Wenn wir auf

`"<em>Test</em>"`

die HTML-Tags matchen wollen:

- `r"<.+>"` matcht den ganzen String **greedy**
- `r"<.+?>"` matcht nur `"<em>"` und `"</em>"` **lazy**

Reguläre Ausdrücke

Heinrich Heine
Universität Düsseldorf

Lazy & Greedy

Greedy Quantoren:

- `r".*"` matcht alles
- `r".+"` matcht auf mindestens ein Zeichen
- `r".?"` matcht auf höchstens ein Zeichen
- `r".{n}"` matcht auf genau n Zeichen
- `r".{n,}"` matcht auf mindestens n Zeichen
- `r".{n,m}"` matcht auf n bis m Zeichen

Lazy & Greedy

Lazy Quantoren:

- `r".*?"` matcht 0 Zeichen oder mehr
- `r".+?"` matcht auf mindestens ein Zeichen
- `r".??"` matcht auf höchstens ein Zeichen
- `r".{n}?"` matcht auf genau n Zeichen
- `r".{n,}?"` matcht auf mindestens n Zeichen
- `r".{n,m}?"` matcht auf n bis m Zeichen

Lazy & Greedy

Lazy Quantoren sind oft effizienter:

Statt mit `r".*domain.com"` erst den ganzen String mit dem `r".*"` zu matchen und dann via Backtracking `"domain.com"` zu matchen,

wird `r".*?domain.com"` zunächst alles bis zum ersten `"d"` matchen und dann schauen, ob es reicht.

Lazy & Greedy

Um zu verstehen,

Ob lazy oder greedy effizienter matchen, muss man verstehen wie der endliche Automat arbeiten wird, den der reguläre Ausdruck beschreibt.

Reguläre Ausdrücke

Heinrich Heine
Universität Düsseldorf

Beispiel

`r"^<([a-z]+?)>(.*)</([a-z]+?)>$"`

matcht auf `"<p>html code</p>"` mit Capture groups

1. `"p"`
2. `"html code"`
3. `"p"`

Reguläre Ausdrücke

Heinrich Heine
Universität Düsseldorf

Beispiel

```
r"^<([a-z]+?)>(.*)</[a-z]+?>$"
```

matcht **nicht** auf

```
"<a href=\"url\">text</a>"
```

Reguläre Ausdrücke

Heinrich Heine
Universität Düsseldorf

Tricks: negative lookahead

```
r"(?!23)\d\d"
```

matcht **nicht** auf `"23"`

aber auf `"22"`

Negativer Lookahead konsumiert keine Zeichen!

Reguläre Ausdrücke

Heinrich Heine
Universität Düsseldorf

Tricks: backreferences

```
> pattern = r".*?\b(\w+)\s+\1\b.*"
> regex = re.compile(pattern)
> s = "Sie ist die die beste"
> match = regex.match(s)
> match[1]
'die'
```

Reguläre Ausdrücke

Heinrich Heine
Universität Düsseldorf

Üben

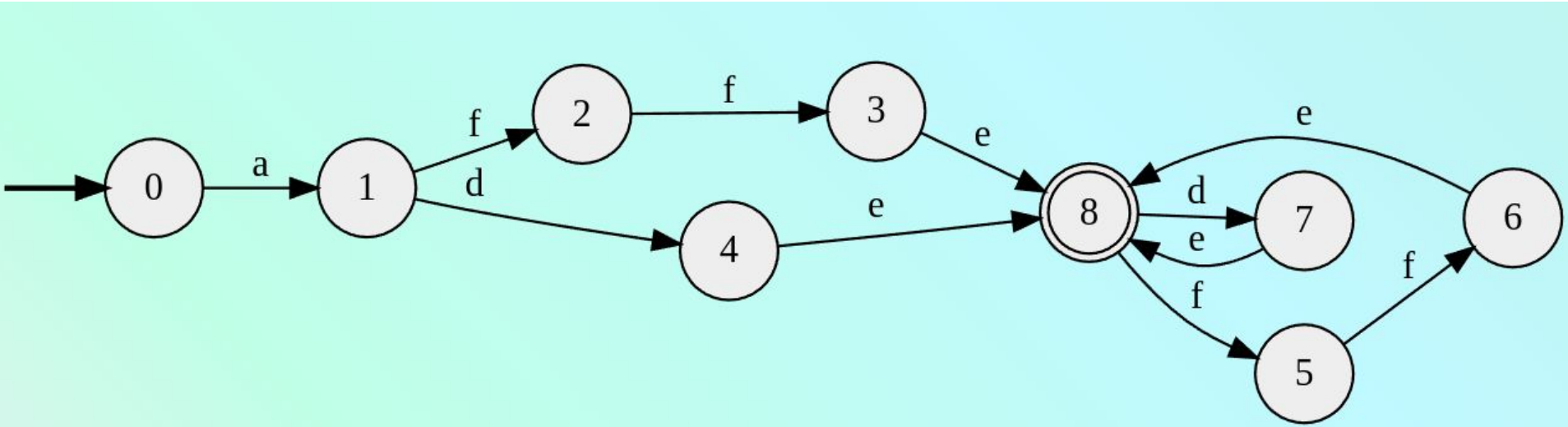
Bei regexone.com kann man interaktiv üben, vorgegebene Strings zu matchen, also dafür Patterns zu schreiben.

Der 27t.github.io/regex-visualizer visualisiert den endlichen Automaten, den ein regulärer Ausdruck beschreibt.

Reguläre Ausdrücke

27t.github.io/regex-visualizer

a(ffe|de)+



Zeit

in passende Datentypen konvertieren



“Time flies by like a Dali painting”

Zeit

Mit Pandas

```
pd.to_datetime(df['day'],
  format = '%Y-%m-%dT%H:%M:%SZ',
  errors = 'coerce')
```

coerce sorgt dafür, dass bei Problemen ein Pandas `NaT` gewählt wird - “not a time”.

Zeit

Mit Pandas

```
pd.to_datetime(df['day'],
 yearfirst = True)
```

Yearfirst interpretiert “10/11/12” als 2010-11-12

Zeit

Mit Pandas

```
pd.to_datetime(df['day'],
 infer_datetime_format = True)
```

Der erste nicht-NaN-Eintrag liefert Pandas eine Hypothese zum Format, danach wird schneller geparst.

Zeit

Mit Pandas

```
df['date'] =
  pd.to_datetime(df['rawdate'])

df['month'] = df['date'].dt.month

df['dayname'] = df['date'].dt.weekday_name
```

Zeitdeltas

Mit Pandas

```
date = pd.to_datetime("24th of
April, 2020")
date + pd.to_timedelta(1, unit='D')
```

Das ist einen Tag später.

Zeit-Ranges

Mit Pandas

```
pd.date_range(start="2022-01-02",
              end="2022-02-05",
              freq='D')
```

Das ist ein `DatetimeIndex`-Objekt.

Zeit-Ranges

Mit Pandas

Der Pandas Datentyp `Timestamp` ist ein Ersatz für Python's eigene `datetime.datetime`.

Zeit

Mit Pandas

```
timestamp =
 pd.Timestamp.utcfromtimestamp(1584199972)
print(timestamp)
```

```
Timestamp('2020-03-14 15:32:52')
```

Konvertiert POSIX Zeitstempel nach Pandas Timestamp

Zeitzonen

Mit Python’s datetime Bibliothek

```
from datetime import timezone
from dateutil import tz
now = datetime.now(tz=tz.tzlocal())
```

Eine Zeitzone ist definiert durch einen fixen Offset von UTC (Universal Time Coordinated)

Zeitzone

Mit Pandas

```
timestamp =
 pd.Timestamp('2020-03-14T15:32:52.192548651',
              tz='UTC')
timestamp.tz_convert(tz='Europe/Berlin')
```

Achtung: nur für timezone-aware timestamps!

Zeitzone

Mit Pandas

```
timestamp =
 pd.Timestamp('2020-03-14T15:32:52.192548651')
timestamp.tz_localize(tz='Europe/Berlin')
```

Macht aus einem timezone-unaware-timestamp einen Zeitstempel, der seine Zeitzone kennt.

Zeitzone

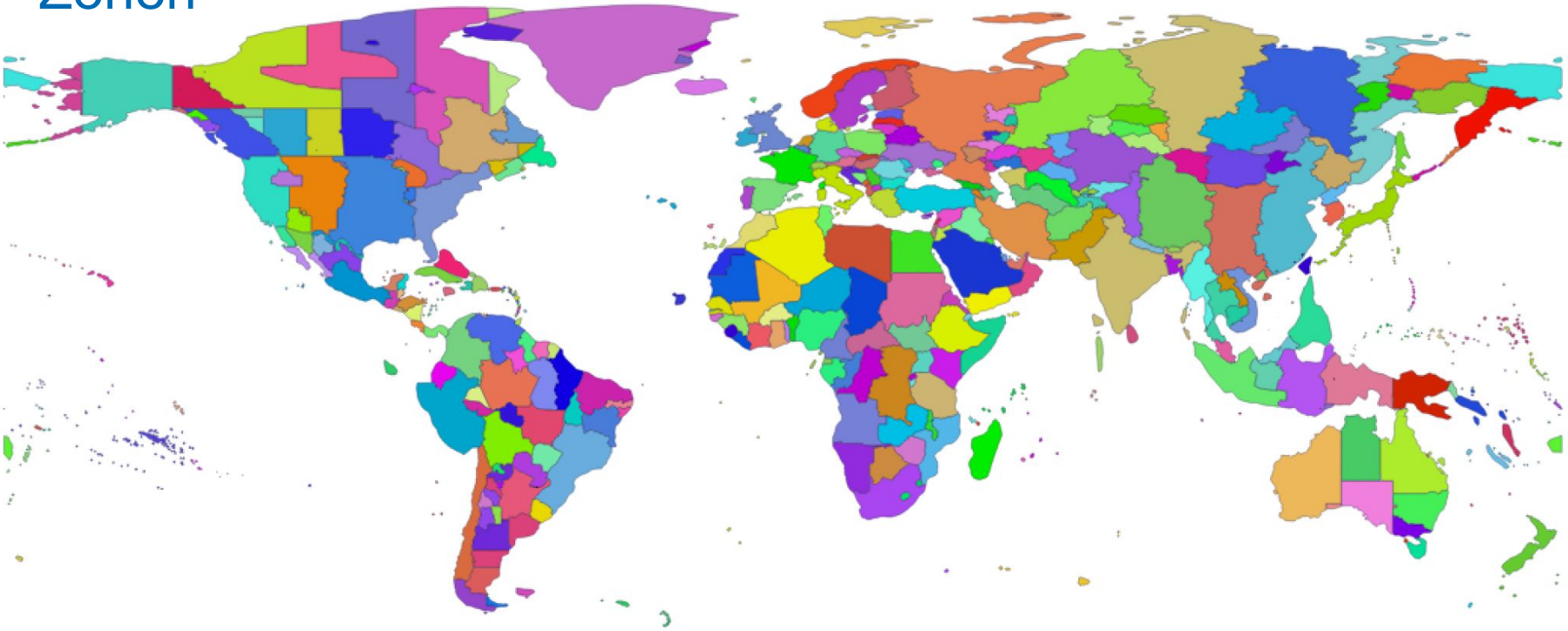
Mit IANA

Die IANA Zeitzonendatenbank (auch Olson oder tzdata genannt) enthält eine Zuordnung Namen->Offsets für Zeitzonen.

en.wikipedia.org/wiki/List_of_tz_database_time_zones

Zeitzone

Zonen



CC-BY-SA Wikipedia user Evansiroky

Danke für Ihre Aufmerksamkeit.